

Module 07 [Java Basics]

Topic 1 [java Programming private method]

Problem Statement: Verify that private methods can be used only inside the class but it cannot be used from outside the class.

Code:

```
class A
{
    private void show()
    {
        System.out.println("Private Method");
    }
    public void display()
    {
        System.out.println("Withen class");
        show();
    }
}

public class Topic1
{
    public static void main(String[] args)
    {
        A a = new A();
        a.display();
        //a.show();
    }
}
```

Topic 2 [java Programming static method/member]

Problem Statement: Verify that static methods can access static method/member but cannot access to non-static method/member.

Code:

```
class A
{
    public static int x = 10;
    int y = 20;
    void display()
```

```

    {
        System.out.println("Non static: "+ y);
    }

    public static void show()
    {
        System.out.println("Static method: "+x);
    }
}
public class Topic2
{
    public static void main(String[] args)
    {
        A a = new A();
        a.display();
        a.show();
    }
}

```

Topic 3 [java Programming: Anonymous object]

Problem Statement: Write a Java program that creates a Point object using anonymous object using the following getPoint() method.

Point pt=getPoint();

Code:

```

class Point {
    int x;
    int y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    void showPoint() {
        System.out.println("X = " + x + ", Y = " + y);
    }
}

public class Mainpoint {

    static Point getPoint() {
        return new Point(5, 10);
    }

    public static void main(String[] args) {

```

```

        getPoint().showPoint();
    }
}

```

Topic 4 [java Programming: ArrayList<> object]

Problem Statement: Write a Java program performs the following operations using ArrayList Object

- i) Declare a ArrayList object ax to store flowers
- ii) Add three flowers: Rose, Lilie, Carnation, Daisie, and Tulip
- iii) Insert a new flower Peonie in the second position of ax
- iv) Delete the flower Lilie
- v) Find any flower in ax
- vi) Sort the list
- vii) Delete the list

Code:

```

import java.util.*;
public class Topic2
{
    public static void main(String[] args)
    {
        ArrayList<String> ax = new ArrayList<>();
        ax.add("Rose");
        ax.add("Lilie");
        ax.add("Carnation");
        ax.add("Daisie");
        ax.add("Tulip");

        System.out.println("Initial Array: "+ax);

        ax.add(1,"Peonie");
        System.out.println("After Insertion: "+ ax);

        ax.remove("Lilie");
        System.out.println("After Deletion: "+ ax);

        String search = "Tulip";
        if(ax.contains(search))
        {
            System.out.println(search+"found in the list");
        }
        else
        {
            System.out.println(search + "not found");
        }
    }
}

```

```

        Collections.sort(ax);
        System.out.println("Sorted list"+ ax);

        ax.clear();
        System.out.println("After Clearing: "+ ax);
    }
}

```

Topic 5 [inheritance: use of super keyword]

Problem Statement: For the following Java program write

- i) to display x of class A in class B
- ii) statement to call getX() of class A in class B
- iii) to call parameterized constructor of in class B

<pre> class A{ int x; public A(){ x=0; } public A(int x){ this.x=x; } </pre>	
<pre> public int getX(){ return(x+10); } class B extends A{ int x=20; public int getX(){ return(x+10); } } public class First{ public static void main(String[] args) { </pre>	

Code:

```

class A
{
    int x;
    public A()
    {
        x=0;
    }
    public A(int x)

```

```

    {
        this.x=x;
    }
    public int getX()
    {
        return(x+10);
    }
}
class B extends A
{
    int x=20;
    public int getX()
    {
        return(x+10);
    }
    public int getXa()
    {
        return super.x;
    }
    public void show()
    {
        System.out.println(super.getX());
    }
    B(int x)
    {
        super(x);
        System.out.println("Super Parameterized Constructor");
    }
}
public class First
{
    public static void main(String[] args)
    {
        B b = new B(10);
        System.out.println(b.getXa());
        System.out.println(b.getX());
        b.show();
    }
}

```

Topic 6 [interface in Java]

Problem Statement: Write three interfaces in Java namely AI, BI and CI with the following activities:

- i) Declare method PrintA() in AI
- ii) Declare method PrintB() in BI
- iii) Declare method PrintC() in CI
- iv) write body of PrintA() in class A
- v) write body of PrintB() in class B which is inherited from class A
- vi) write body of PrintC() in class C which is inherited from class B
- vii) Create object of class C in main() of First class and invoke PrintA(), PrintB() and PrintC()

Code:

```
interface AI
{
    void printA();
}
interface BI
{
    void printB();
}
interface CI
{
    void printC();
}
class A implements AI
{
    @Override
    public void printA()
    {
        System.out.println("Print A");
    }
}
class B extends A implements BI
{
    public void printB()
    {
        System.out.println("Print B");
    }
}
class C extends B implements CI
{
    public void printC()
    {
        System.out.println("Print C");
    }
}
```

```
    }  
}  
public class interfacejava  
{  
    public static void main(String[] args)  
    {  
        C obj = new C();  
        obj.printA();  
        obj.printB();  
        obj.printC();  
    }  
}
```